

BabyShopHub

Windows Setup Guide — From a Clean PC to a Running App

Aptech eProject Submission

Version 1.0

Table of Contents

Before you start.....	2
Step 1 — Open PowerShell.....	3
Step 2 — Install Git.....	4
Step 3 — Install the Flutter SDK.....	5
3a. Create the folder and download Flutter.....	5
3b. Tell Windows where Flutter is (add it to PATH).....	5
Step 4 — Install the tools Flutter needs (run flutter doctor).....	7
Step 5 — Install the Windows build toolchain.....	8
Step 6 — Install Visual Studio Code.....	9
Step 7 — Put the project on your PC.....	10
Step 8 — Generate the platform files.....	11
Step 9 — Download the project's dependencies.....	12
Step 10 — Run the app.....	13
Step 11 — Log in and explore.....	14
Step 12 — Running it again next time.....	15
Appendix A — Running on an Android phone (optional).....	16
Option 1 — A real Android phone (simplest).....	16
Option 2 — An Android emulator (no physical phone).....	16
Clearing the Android checklist.....	16
Appendix B — Quick command reference.....	17

Before you start

This guide takes a **Windows PC with nothing installed** and ends with the BabyShopHub app running on your screen. You do not need any programming experience. Every step below shows you the **exact command to type** and the **output you should expect to see**, so you always know whether a step worked before you move on.

Read this box once, then begin at Step 1.

How to read this guide

- Text in a grey box that you should type is a *command*. Type it (or copy it) and press **Enter**.
- Under most commands there is an “*Expected output*” box. Your screen will not match it word for word — version numbers and dates differ — but the **shape** should match. If instead you see the word error, stop and read the “*If it goes wrong*” note for that step.
- A command runs inside a **terminal**. On Windows the terminal is an app called **PowerShell**. Step 1 shows you how to open it.

Roughly how long this takes: 30 to 60 minutes, most of which is waiting for downloads. The Flutter download alone is about 1 GB.

What you will install

1. Git — a tool Flutter needs in the background.
2. The Flutter SDK — the toolbox that builds and runs the app.
3. Visual Studio Code — the editor you will run the app from.
4. One “build toolchain” so the app can open as a **Windows desktop window**.

We deliberately run the app on **Windows desktop** first. It is the fastest and most reliable way to see the finished app, and it needs no phone and no Android emulator. Running on an Android phone is covered later in *Appendix A* for when you want it.

Step 1 — Open PowerShell

Click the **Start** button, type powershell, and click **Windows PowerShell** in the results.

A dark blue or black window opens with a blinking cursor. This is your terminal. Every command in this guide is typed here.

To confirm it is working, type this and press **Enter**:

```
echo Hello
```

Expected output

```
Hello
```

If you saw Hello, PowerShell works. Leave this window open — you will use it throughout.

Tip: You can paste into PowerShell with a **right-click** (Ctrl+V may not work). Copy a command from this document, right-click in the PowerShell window, and it pastes.

Step 2 — Install Git

Flutter uses Git behind the scenes. Install it first.

1. In your web browser, go to **<https://git-scm.com/download/win>**.
2. The download of “64-bit Git for Windows Setup” starts automatically. If it does not, click the **64-bit Git for Windows Setup** link.
3. Open the downloaded file. An installer opens.
4. Click **Next** on every screen to accept the defaults, then **Install**, then **Finish**. The defaults are correct — you do not need to change anything.

Now confirm it installed. Close PowerShell and open it again (this is important — a fresh PowerShell picks up the newly installed Git). Then type:

```
git --version
```

Expected output

```
git version 2.46.0.windows.1
```

Your numbers may differ (for example 2.45 or 2.47). As long as you see the words `git version` followed by numbers, Git is installed.

If it goes wrong — if you see `git : The term 'git' is not recognized`, the fresh PowerShell window did not pick up Git. Close **all** PowerShell windows, open a new one, and try `git --version` again. If it still fails, restart the PC and retry.

Step 3 — Install the Flutter SDK

The Flutter SDK is the toolbox that turns the project's code into a running app. We will put it in a simple folder, C:\src\flutter, to avoid the two mistakes that trip people up: spaces in the path, and needing administrator rights.

3a. Create the folder and download Flutter

Copy these three lines into PowerShell one at a time, pressing **Enter** after each. The middle line is a large (~1 GB) download and can take several minutes — wait for the cursor to return before typing the next line.

```
mkdir C:\src
```

```
cd C:\src
```

```
git clone https://github.com/flutter/flutter.git -b stable
```

Expected output (of the third command)

```
Cloning into 'flutter'...
```

```
remote: Enumerating objects: 512345, done.
```

```
remote: Counting objects: 100% (1234/1234), done.
```

```
remote: Compressing objects: 100% (678/678), done.
```

```
Receiving objects: 100% (512345/512345), 210.34 MiB | 8.20 MiB/s, done.
```

```
Resolving deltas: 100% (400000/400000), done.
```

```
Updating files: 100% (8500/8500), done.
```

The percentages count up while it downloads. When you see your normal PS C:\src> prompt again, it is done.

3b. Tell Windows where Flutter is (add it to PATH)

Right now only the C:\src\flutter folder knows about Flutter. This step lets you type flutter from anywhere. Copy this **whole block** into PowerShell and press **Enter**:

```
$flutterBin = "C:\src\flutter\bin"
```

```
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
```

```
if ($userPath -notlike "$flutterBin*") {
```

```
    [Environment]::SetEnvironmentVariable("Path", "$userPath;$flutterBin", "User")
```

```
    Write-Host "Flutter added to PATH."
```

```
} else {
```

```
    Write-Host "Flutter already on PATH."
```

```
}
```

Expected output

```
Flutter added to PATH.
```

Now close PowerShell and open a new one so the change takes effect. Confirm it worked:

```
flutter --version
```

Expected output

```
Flutter 3.24.0 • channel stable • https://github.com/flutter/flutter.git  
Framework • revision 80c2e84975 (3 weeks ago) • 2026-08-28 10:00:00 -0700  
Engine • revision b8800d88be  
Tools • Dart 3.5.0 • DevTools 2.37.2
```

The version numbers will differ. The key is that you see **Flutter**, a version number, and a **Dart** version on the last line. The very first time you run flutter, it may pause for a minute to finish setting itself up — that is normal.

If it goes wrong — flutter : The term 'flutter' is not recognized means the PATH change has not taken effect. Close all PowerShell windows, open a new one, and retry. If it persists, restart the PC.

Step 4 — Install the tools Flutter needs (run flutter doctor)

Flutter ships a self-check called **doctor** that tells you exactly what is still missing. Run it:

```
flutter doctor
```

Expected output (on a fresh PC, before you install anything else)

Doctor summary (to see all details, run flutter doctor -v):

[√] Flutter (Channel stable, 3.24.0, on Microsoft Windows [Version 10.0.26200])

[!] Windows Version (Installed version of Windows is version 10 or higher)

[X] Android toolchain - develop for Android devices

 X Unable to locate Android SDK.

[X] Visual Studio - develop for Windows

 X Visual Studio not installed; this is necessary to develop Windows apps.

[!] Android Studio (not installed)

[√] VS Code (not installed)

[!] Connected device

 ! No devices available

Do not be alarmed by the red [X] marks — a fresh PC always shows them. Each one is a checklist item. For this guide you only need to fix **one** of them: *Visual Studio — develop for Windows*. That is Step 5.

You can ignore the Android [X] for now. You only need it if you want to run on an Android phone (see *Appendix A*).

Step 5 — Install the Windows build toolchain

To build the app as a Windows desktop window, Flutter needs Microsoft's C++ build tools. This is the single most important [X] to clear.

1. Go to **<https://visualstudio.microsoft.com/downloads/>**.
2. Scroll to **Tools for Visual Studio** and, under **Build Tools for Visual Studio 2022**, click **Download**. (This is the free, smaller package — you do **not** need the full Visual Studio IDE.)
3. Run the downloaded installer. After it loads, it shows a grid of “workloads”.
4. Tick the box for **Desktop development with C++**.
5. Click **Install** (bottom-right). This downloads several GB and takes a while. Let it finish, then restart the PC if it asks.

Confirm it worked. Open a new PowerShell and run doctor again:

```
flutter doctor
```

Expected output (the Visual Studio line has turned green)

```
[√] Flutter (Channel stable, 3.24.0, on Microsoft Windows [Version 10.0.26200])
[√] Visual Studio - develop for Windows (Visual Studio Build Tools 2022 17.11.0)
[X] Android toolchain - develop for Android devices
[√] VS Code (not installed)
[!] Connected device
```

As long as **Visual Studio** now shows a green [√], you are ready to run the app. The Android [X] can stay red.

Step 6 — Install Visual Studio Code

Visual Studio Code (VS Code) is the editor you will open the project in and press “run” from.

1. Go to **<https://code.visualstudio.com>** and click the big blue **Download for Windows** button.
2. Run the installer. Click **Next** through the screens. On the “Select Additional Tasks” screen, make sure **Add to PATH** is ticked (it is by default). Finish the install.
3. Open VS Code.
4. On the left edge, click the **Extensions** icon (four squares, the last one with one square pulling away).
5. In the search box type **Flutter**. The top result is **Flutter** by fluttertools. Click **Install**. (This automatically installs the Dart extension too.)

You now have everything installed. The rest of the guide is about opening and running the actual BabyShopHub project.

Step 7 — Put the project on your PC

You were given the project as a folder named **BabyShopHub** (inside a ZIP file). Do this:

1. Find the BabyShopHub.zip file (likely in your **Downloads** folder).
2. Right-click it and choose **Extract All...**, then **Extract**.
3. Move the extracted **BabyShopHub** folder somewhere simple with no spaces in the path, for example directly onto your C: drive, so it lives at **C:\BabyShopHub**.

Confirm PowerShell can see it:

```
cd C:\BabyShopHub
```

```
dir
```

Expected output

Directory: C:\BabyShopHub

Mode	LastWriteTime	Length	Name
d----	2026-09-21 10:00		assets
d----	2026-09-21 10:00		docs
d----	2026-09-21 10:00		lib
d----	2026-09-21 10:00		tool
-a----	2026-09-21 10:00	1024	analysis_options.yaml
-a----	2026-09-21 10:00	1100	pubspec.yaml

You should see the folders **lib**, **assets**, **docs**, **tool** and the file **pubspec.yaml**. If you do, you are in the right place. Stay in this folder for every remaining step.

If it goes wrong — if `cd C:\BabyShopHub` says Cannot find path, the folder is somewhere else or has a different name. Open File Explorer, find the folder that contains `pubspec.yaml`, click the address bar to see its full path, and use that path after `cd`.

Step 8 — Generate the platform files

The project you were given contains the **source code** (the lib folder) and its **images** (the assets folder), but not the Windows/Android “wrapper” files — those are generated on your own machine so they match your exact Flutter version. This one command creates them, and it does **not** touch or overwrite the app’s code:

```
flutter create .
```

(The dot at the end means “here, in this folder”. Do not leave it out.)

Expected output

```
Recreating project ....
  Running "flutter pub get" in BabyShopHub...
Wrote 78 files.
```

All done!

In order to run your application, type:

```
$ cd .
$ flutter run
```

Your application code is in .\lib\main.dart.

This has now created windows, android, and a few other folders next to your lib folder. Your app code was left untouched.

Step 9 — Download the project's dependencies

The app uses a few free add-on packages (for the local database, password security, and so on). This command reads the `pubspec.yaml` file and downloads exactly those:

```
flutter pub get
```

Expected output

```
Resolving dependencies...  
Got dependencies!
```

You may see a few lines listing package names and version numbers above Got dependencies! — that is normal. As long as the last line is Got dependencies!, every package the app needs is now on your PC.

If it goes wrong — if you see a red message mentioning version solving failed, run `flutter --version` to confirm your Flutter is on the **stable** channel, then run `flutter pub get` again. A second run usually succeeds once the first has warmed the download cache.

Step 10 — Run the app

This is the moment it all comes together. Tell Flutter to run the app as a Windows desktop window:

```
flutter run -d windows
```

The first run compiles everything from scratch and can take **2 to 4 minutes**. You will see a lot of build text scroll past — this is normal and only happens the first time.

Expected output

```
Launching lib\main.dart on Windows in debug mode...  
Building Windows application...  
✓ Built build\windows\x64\runner\Debug\babysophub.exe
```

Flutter run key commands.

r Hot reload.

R Hot restart.

h List all available interactive commands.

d Detach (terminate "flutter run" but leave application running).

c Clear the screen

q Quit (terminate the application on the device).

Running with sound null safety

A **window titled BabyShopHub opens** showing a pink splash screen, then the login screen. **That is the finished app running on your PC.**

Leave the PowerShell window open while you use the app. When you are finished, click back on PowerShell and press **q** to quit, which also closes the app window.

If it goes wrong

- If it says No Windows desktop project configured, you skipped Step 8. Run `flutter create .` then try again.
- If it says Building with plugins requires symlink support or mentions *Developer Mode*, run `start ms-settings:developers`, turn **Developer Mode On** in the window that opens, then run `flutter run -d windows` again.
- If the very first build fails but the second works, that is a known first-build quirk — just run the command again.

Step 11 — Log in and explore

The app comes pre-loaded with sample data (products, categories, and two ready accounts) so you can explore immediately without setting anything up.

Two demo accounts are provided. They are also shown on the login screen for convenience — you can tap them to auto-fill.

Role

Customer (shopper)

Administrator (shop manager)

Log in as the customer to browse products, search, add items to the cart, and place an order with the dummy payment screen.

Log in as the administrator to see the management side: the dashboard, orders, products, categories, users, reviews, and the support inbox.

You can also tap **Create account** on the login screen to register a brand-new customer of your own — that exercises the registration flow.

The passwords follow the app's own rule: at least 8 characters with at least one letter and one digit. If you register a new account, your password must meet that rule too.

Step 12 — Running it again next time

Once everything is installed, coming back to the app is quick. You do **not** repeat Steps 1–9. Just open PowerShell and run:

```
cd C:\BabyShopHub
```

```
flutter run -d windows
```

The app opens again. That is all.

If you ever want a completely fresh start — wiping the local database back to the original sample data — close the app, delete the file the app created, and run again. The app rebuilds the sample data automatically on next launch. (The database file lives in your user AppData folder; the app recreates it if it is missing.)

Appendix A — Running on an Android phone (optional)

Windows desktop is the easiest way to demo the app. If you specifically need it on an Android phone or emulator, do the following **after** completing Steps 1–9.

Option 1 — A real Android phone (simplest)

1. On the phone, open **Settings** → **About phone** and tap **Build number** seven times to unlock **Developer options**.
2. In **Settings** → **Developer options**, turn on **USB debugging**.
3. Connect the phone to the PC with a USB cable. On the phone, tap **Allow** on the “Allow USB debugging?” prompt.
4. Confirm the PC sees it:
flutter devices

Expected output (your phone appears in the list)

2 connected devices:

SM G991B (mobile) • R5CN... • android-arm64 • Android 14 (API 34)
Windows (desktop) • windows • windows-x64 • Microsoft Windows

5. Run on the phone:
flutter run -d android

The first build is slower than desktop. When it finishes, the app opens on the phone.

Option 2 — An Android emulator (no physical phone)

This needs **Android Studio**, which is a large extra install.

1. Download and install Android Studio from <https://developer.android.com/studio>.
2. Open it once and let it finish “downloading components”.
3. Go to **More Actions** → **Virtual Device Manager**, click **Create Device**, pick a phone such as **Pixel 7**, and download a system image (for example **Tiramisu / API 33**). Finish, then press the green **play** button to start the emulator.
4. With the emulator running, back in PowerShell:
flutter run -d emulator

Clearing the Android checklist

After installing Android Studio, accept the Android licences so flutter doctor goes green:

flutter doctor --android-licenses

Press **y** and **Enter** at each prompt. Then flutter doctor should show a green [✓] for the Android toolchain.

Appendix B — Quick command reference

Once set up, these are the only commands you will ever need. Run them from inside the C:\BabyShopHub folder.

I want to...

Run the app on Windows
Run on a connected phone
See what devices are available
Re-download packages (rarely)
Check my install is healthy
Check the code for errors
Stop the running app
Reload code changes instantly

A note on flutter analyze. This command reads the whole project and reports any code problems. On this project it should finish with No issues found!. It is the authoritative check that the code is sound on your machine, and it is worth running once after setup:

flutter analyze

Expected output

Analyzing BabyShopHub...
No issues found!

End of Windows Setup Guide. If every step's output matched, BabyShopHub is installed and running. The next document, the User Guide, walks through actually using the app as both a shopper and an administrator.